Solution: Plus One

Problem Statement

You are given a `digits` array, where `digits[i]` is between `0` to `9`. It is given that when you combine all elements of the `digits` array in `left` to `right` order, it forms the `large integer`.

Add `1` to `large integer` and return the array containing the digits of the resultant `large` integer.

Examples

- **Example 1**:
  - **Input**: `[1, 2, 3]`
  - **Expected Output**: `[1, 2, 4]`
  - **Justification**: The number 123 is incremented to 124. No carry over beyond the last digit.
- **Example 2**:
  - **Input**: `[4, 9, 9, 9]`
  - **Expected Output**: `[5, 0, 0, 0]`
  - **Justification**: Incrementing 4999 results in a carry over at each digit, leading to 5000.
- **Example 3**:
  - **Input**: `[9, 8, 7, 6, 8, 9, 9, 9]`
  - **Expected Output**: `[9, 8, 7, 6, 9, 0, 0, 0]`
  - **Justification**: Incrementing the last three digits of 98768999 by 1 results in a carry that changes these digits to 0 and increases the 5th digit by 1, resulting in 98769000.

Solution

To solve this problem, we'll iterate through the array of digits from the end towards the beginning, mimicking the process of addition in arithmetic. This approach works because incrementing by one affects only the least significant digits and may carry over to the more significant digits in a cascading manner. By starting from the end, we can efficiently manage carries and directly modify the array in place, if possible. This method is effective as it directly mirrors the manual process of addition, ensuring that all edge cases, such as carry-overs that increase the array size, are handled naturally within the algorithm's flow.

The key to this approach is handling the carry-over. If incrementing the current digit results in a value less than 10, we can stop, as no further carry will occur. However, if it equals 10, we set the current digit to 0 and carry over to the next significant digit. This process continues until we either no longer have a carry or reach the most significant digit. If we still have a carry at the most significant digit, we need to adjust the array size to accommodate the increased magnitude of the number.

Step-by-step Algorithm

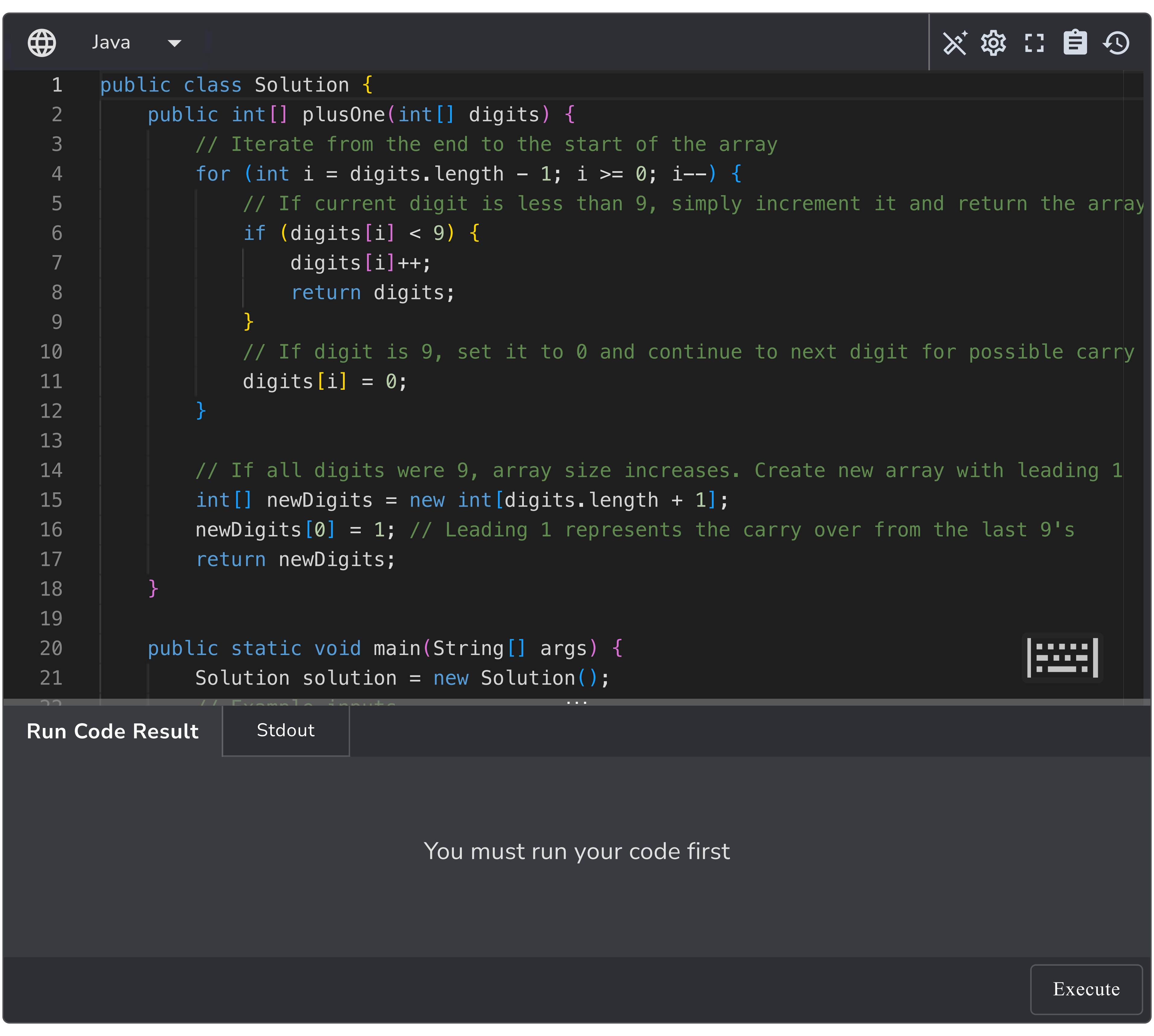
- Start iterating from the last digit of the array towards the first.
- Increment the last digit by 1. If the result is less than 10, update the digit and return the array.
- If the digit becomes 10 after incrementing, set it to 0 and carry over 1 to the next significant digit.
- Continue this process moving towards the more significant digits.
- If a carry is present when we reach the most significant digit and it results in 10, prepend 1 to the array to accommodate the carry.
- Return the modified array as the final result.

Algorithm Walkthrough

Using the input `[9, 8, 7, 6, 8, 9, 9, 9]`, let's illustrate the algorithm step by step:

1. **Start with the last digit (9)**:
   - Incrementing 9 by 1 gives 10. Set this digit to 0, carry over 1.
2. **Move to the next digit to the left (second 9)**:
   - Again, 9 + 1 = 10. Set this digit to 0, carry over 1.
3. **Proceed to the next digit to the left (third 9)**:
   - Similarly, 9 + 1 = 10. Set this digit to 0, carry over 1.
4. **Next, address the fourth digit from the right (8)**:
   - This time, 8 + 1 = 9. Update this digit to 9. Since the result is less than 10, no further carry is needed.
5. **Since we no longer have a carry**, the process stops. The digits to the left of the one we just updated remain unchanged.
6. **The final array**, after processing the increment and handling all necessary carry-overs, is `[9, 8, 7, 6, 9, 0, 0, 0]`.

Code



```java
public class Solution {
    public int[] plusOne(int[] digits) {
        // Iterate from the end to the start of the array
        for (int i = digits.length - 1; i >= 0; i--) {
            // If current digit is less than 9, simply increment it and return the array
            if (digits[i] < 9) {
                digits[i]++;
                return digits;
            }
            // If digit is 9, set it to 0 and continue to next digit for possible carry
            digits[i] = 0;
        }

        // If all digits were 9, array size increases. Create new array with leading 1
        int[] newDigits = new int[digits.length + 1];
        newDigits[0] = 1; // Leading 1 represents the carry over from the last 9's
        return newDigits;
    }

    public static void main(String[] args) {
        Solution solution = new Solution();
```

Complexity Analysis

Time Complexity

- $O(n)$: The algorithm iterates over each digit of the input array exactly once in the worst-case scenario (when all digits are 9 and need to be turned into 0, with an additional digit added at the beginning). Here, n represents the number of digits in the input array.

Space Complexity

- $O(1)$: Algorithm performs the in-place addition. So, it doesn't require extra space.

← Previous
Plus One (Easy)

Next →
Intersection Of Two Arrays (Easy)

☐ Mark as Completed